Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology

School of Computing

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

ACADEMIC YEAR 2025 – 2026 (Summer Semester)

10211CA207-Database Management System

NAME: N. mohan venkat

VTU NO: 30539

REG.NO: 24UECL0035

BRANCH: CSE (AI & ML)

YEAR/SEM: 2nd year, 3rd sem

SLOT: S10L12

Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology

NAAC A++ | CATEGORY-1 | UGC

School of Computing
Department of Computer Science & Engineering
(Artificial Intelligence and Machine Learning)

ACADEMIC YEAR 2025-26 (SUMMER SEMESTER)

BONAFIDE CERTIFICATE

NAME	: N.mohan venkat	**BRANCH**	: CSE (AI & ML)
VTU NO.	: 30539	**REG.NO.**	: 24UECL0035
YEAR/SEM	: 2nd year, 3rd sem	**SLOT NO.**	: S10 L12

Certified that this is a bonafide record of work done by above student in the **"10211CA207-DATABASE MANAGEMENT SYSTEMS LABORATORY"** during the year 2025-2026 (Summer Semester).

R.T. 30/10/25 | 30/10/25

SIGNATURE OF LAB HANDLING FACULTY | **SIGNATURE OF HOD**

Submitted for the Semester Practical Examination held on **04.11.25** at Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and Technology.

INTERNAL EXAMINER | **EXTERNAL EXAMINER**

INDEX

S.No	Date	Title	Page No.	Marks	Faculty Signature
1.	24/07/25	Conceptual Design after Formal Technical Review	1-3	18	24/7
2.	31/07/25	Generating design of other traditional database models	4-9	18	31/7
3.	07/08/25	Developing queries with DML Single-row functions and operations	10-14	18	7/8
4.	14/08/25	Developing queries with DML Multi-row functions and operations	15-20	19	14/8
5.	21/08/25	Writing Join Queries, equivalent, and/or recursive queries	21-25	18	21/8
6.	28/08/25	Writing PL/SQL using Procedures, Function	26-29	18	28/8
7.	04/09/25	Writing PL/SQL using Loops	29-31	18	4/9
8.	11/09/25	Normalizing databases using functional dependencies up to BCNF	32-35	18	11/9
9.	18/09/25	Backing up and recovery in databases	36-37	18	18/9
10.	25/09/25	CRUD operations in Document databases	38-40	18	25/9
11.	09/10/25	CRUD operations in Graph databases	41-42	18	9/10
12.	16/10/25	Micro Project: A Gift coupons application that displays offers and payment related information	43-60	18	16/10

Completed

Total Marks: 227/240

Signature of Faculty

Online Temple Ticket Booking management system

Date: 29/07/25

* A Online temple ticket booking management system allows the devotees to book tickets from temple visits, special darshans, poojas and Others Events online, reducing physical Queues and Improving crowd management. These system often Include features like date and time slot selection, payment processing and the Generation of tickets or passes.

* Entity:

A Real-World object or concept that can be distinctly as Identified

Examples Include Students, Courses or products.

* Entity Set:

A collections of Entities of the same type. for Instant, Students In a university would from an Entity Set.

* Attributes:

A property or characteristic of an Entity for Example a Students might have attributes like name, ID and major.

* Relationship: An association or Interaction between two or more Entities for Examples - a Student might Enroll In a course, Establishing a.

Online Temple Ticket Booking management system Date: 24/07/25

* A Online temple ticket booking management system allows the devotees to book tickets from temple visits, special darshans, poojas and Others Events online, reducing physical Queues and Improving crowd management. These system often Include features like date and time slot selection, payment processing and the Generation of Tickets or passes.

Entity:

A Real World object or concept that can be distinctly as Identified

Examples Include Students, Courses or products.

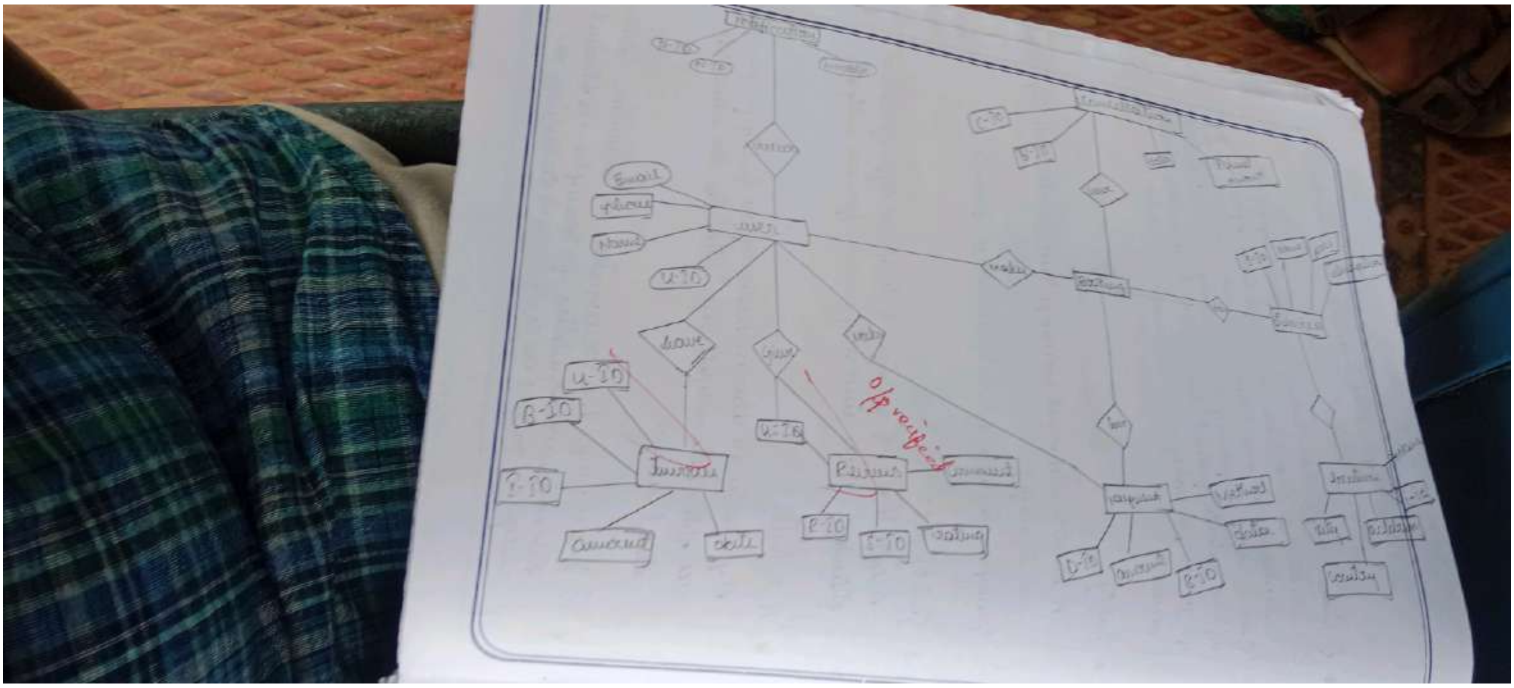
Entity set:

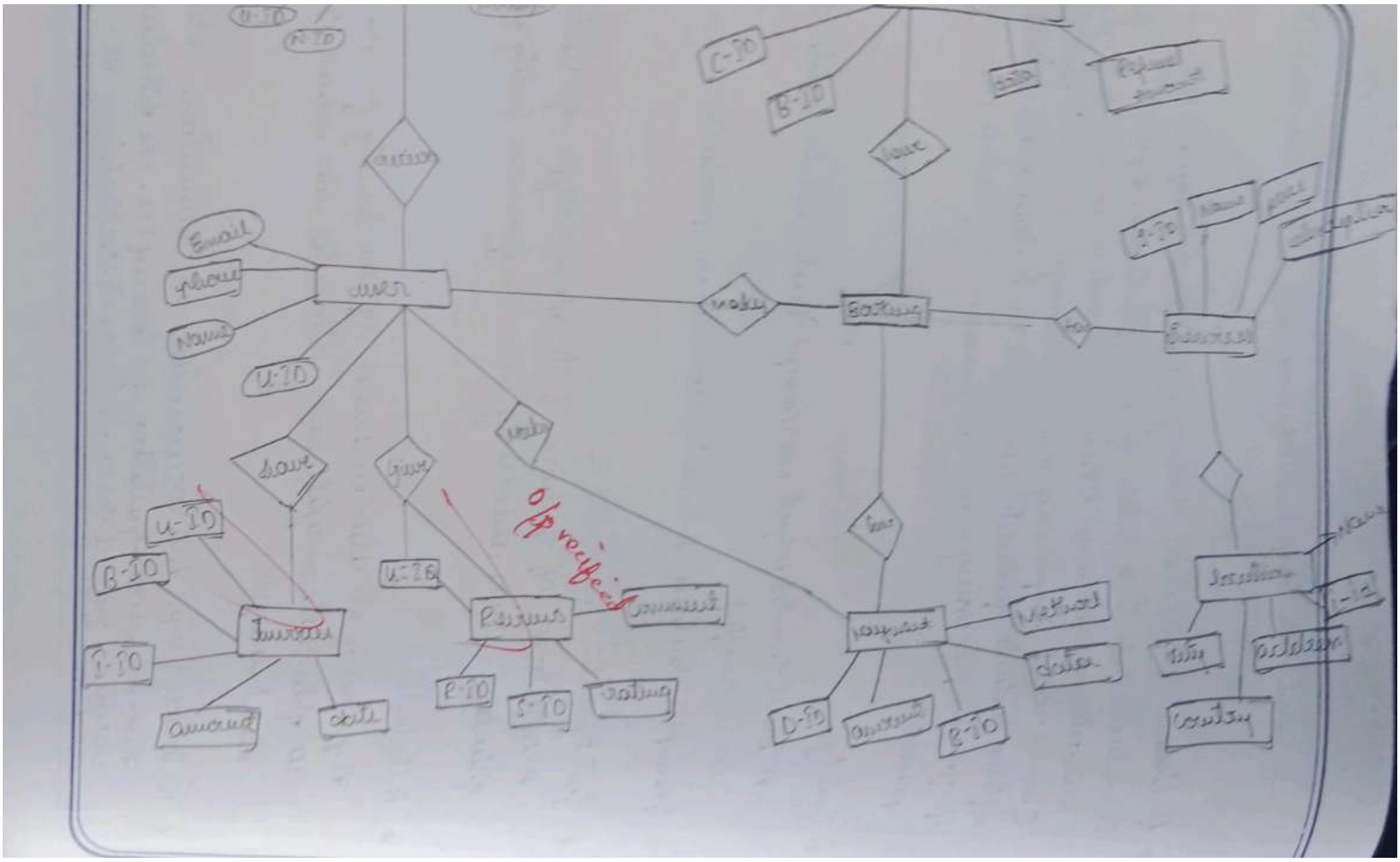
A collection of Entities of the same type. for Instant: Students In a university would form an Entity set.

Attributes:

A property or characteristic of an Entity for Example a Students might have attributes like name, ID. and major.

Relation ship: An association or Interaction between Two or more Entities for Examples - a Student might Enroll In a course, Establishing a.


Email
phone
Name
U-ID
have
give
R-ID
I-ID
amount
date
Review
comment
rating
O/p verified



Email
phone
Name
U-ID
user
make
Booking
save
give
Review
comment
payment
Method
status
D-ID
amount
B-ID
R-ID
rating
date
u-ID
O/P verified
address
country
C-ID

in Relationship between the "Student" "and" "Course" System

* Aim:

To design and develop an Entity-relationship (ER) model for a temple online ticket booking management system that allows the devotees to book tickets for temple visits, special darshans, poojas and other spiritual events reducing physical queues and improving crowd management.

* Attributes:

- devotees: Devote-ID, Name, phone, Email, address, Age, Gender.
- Booking: Booking-ID, date, time-slot, No of persons, Status.
- Events: Events ID, Event-name, description, price, Event-type, duration.
- Temple: Temple-ID, Temple-name, location, Opening hours.
- payment: payment-ID, Amount, method, Status, date.
- Tickets: Ticket-ID, Issue-date, QR-code, Vali
- Admin: Admin-ID, Name, username, password, Role.

* Types of attributes
- Simple : Name, price
- composite : address (street, city, state).
- multi-valued : phone.
- derived : age (from DOB).

* Relationships :
Devotee → Booking (makes)
Booking → Event (for).
Booking → payment (has).
Booking → Tickets [generates].
Event → Temple [hosts]
admin → Event [manages]

* Relationship types :
* One - to - One : Booking → payment, Booking → Tickets.
* One - to - One : Devotee → Booking.
* many to - one : Booking → Event.
* One to many : Temple → Event.
* One - to - Many : Admin → Event.

VEL TECH - CSE	
EX NO	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	4.5
TOTAL (20)	13+5=18
SIGN WITH DATE	

* Cardinality : 1:N & 1:1 where applicable.

Result: The ER diagram for the temple tickets booking management system was successfully designed. Showing all Entities attributes and relationship with correct cardinalities for data base. Implementations

m 2/7/25

Types of attributes
Simple : Name, price.
composite : address (street, city, state).
multi-valued : phone.
derived : age (from DOB).

Relationships :
User → Booking (makes).
Booking → Event (for).
Booking → payment (has).
Booking → Tickets [generates].
Booking → Tickets [hosts]

Enhancing design of [illegible] database model

* Aim: creating conceptual / relational model of the [illegible] by enhancing the [illegible] by performing following tasks using forms of [illegible] rules.

1. Identify the specificity of each relationship, find and remove surplus relation.

* Relationship 1: Temple manages Event (one-to-many).
* Specificity : One temple can organize many Events and each Event is managed by only one Temple.
* Surplus Relation: No surplus relation needed since it is already one-to-many.

* Relationship 2: Event has ticket (one-to-many).
  Specificity : Each Event can have many Ticket types (Eg: General, VIP), but each Ticket type belongs to one Event.
  Surplus Relation: No surplus relation needed: one-to-many is appropriate.

* Relationship 3: Customer books Ticket (many-to-many)
  Specificity: A Customer can be book many Tickets and a Ticket can be booked by many customers (for Group bookings).

[illegible] design of [illegible] conceptual database model

* Aim: Creating conceptual [illegible] model of the [illegible] by [illegible] the [illegible] by performing following tasks using [illegible] rules.

1. Identify the specificity of each relationship, find and form surplus relation.

* Relationship 1: Temple manages Event (one-to-many).
* Specificity: One temple can organize many events but each event is managed by only one temple.
* Surplus Relation: No surplus relation needed since it is already one-to-many.

* Relationship 2: Event has Ticket (one-to-many).
  Specificity: Each Event can have many Ticket Types Eg: General, VIP), but each Ticket Type belongs to one Event.
  Surplus Relation: No surplus relation needed: one-to-many is appropriate.

* Relationship 3: Customer books Ticket (many-to-many)
  Specificity: A customer can be book many tickets and a ticket can be booked by many customers (for Group bookings).

* Surplus Relation: Yes, Surplus relation needed for many-to-many is appropriate.

* Relationship 4: Ticket allocated to Seat (one-to-One).
* Specificity: Each Ticket corresponds to a Specific Seat and Each Seat can be assigned to only one Ticket.

* Surplus Relation: No Surplus relation needed; one-to-one relationship is sufficient.

26. check is-a hierarchy / has-a hierarchy and perform Generalization and/or Specialization

Generalization Example:
* Entities customer, admin.
common attributes, User-Id-first-Name, Email, contact Number, password.
Generalized Superclass: User With above attributes

Subclasses:
customer: additional attributes like customer-ID Address, payment, Info.
Admin: Additional attributes like admin-ID, Role.

Surplus Relation: Yes, Surplus relation needed for many-to-many is appropriate.

Relationship 4: Ticket allocated to seat (one-to-one).

Specificity: Each ticket corresponds to a specific seat and each seat can be assigned to only one ticket.

Surplus Relation: No surplus relation needed; one-to-one relationship is sufficient.

check is-a hierarchy / has-a hierarchy and perform Generalization and/or Specialization

Generalization Example:

Entities customer, admin.

Common attributes, User-Id-first-Name-Email, contact-Number, password.

Generalized Superclass: User with above attributes

Subclasses:

customer: additional attributes like customer-ID, Address, payment, Info.

Admin: Additional attributes like admin-ID, Role

Specialization Examples:-

Entity : Ticket .

Specialized Into :

- General Ticket (attributes price, General-access-area).
- Vip Ticket (attributes price, access-To-Special-area, complementary-Services).

2. (C) find The domine of attributes and perform check constraint.

Attributes	domine	check Constraint Example.
age (customer)	positive Integer	check (age) = 18)
Ticket-price	positive decimal number	check (Ticket-price
booking-data	Data (not In past)	check (booking-d >= current-
Seat-number	String or number. within Vaild range	check (Seat-n between 1

2d. Rename the Relations (Tables).

Example renaming colums for clarity :

Alter Table customer Rename column contact-no To phone

Alter Table Ticket Rename column price To Ticket-pric

Alert Table Booking Rename column book-data To booki data;

Specialization Examples:-

Entity: Ticket.

Specialized Into:

General Ticket (attributes price, general-access-area)

Vip Ticket (attributes price, access-to-special-area, complementary-services).

find the domine of attributes and perform check constraint

Attributes	domine	check constraint Example.
age (customer)	positive Integer	check (age)=18)
Ticket-price	positive decimal number	check (Ticket-price
booking-data	Data (not In past)	check (booking-… >= current
Seat-number	String or number. within valid range	check (seat-… between

Rename the Relations (Tables).

Example renaming colums for clarity:

Alter Table Customer Rename column contact-no to phone

Alter Table Ticket Rename column price To ticket-price

Alert Table Booking Rename column book-data to booking-data;

(1)

2.c Perform SQL Relations using DDL and DCL commands.
DDL (Data Definition Language):

create user Table (Generalization).
create Table User (.
User - ID Int primary key,
first - Name Varchar (50),
last - Name Varchar (50),
Email Varchar (100),
contact - Number Varchar (15),
password Varchar (100),
user - Type Varchar (10) - 'Admin' or 'customer'
);

- Specialized customer Table.
create Table customer (.
customer - ID Int primary key,
User - ID INT.
AGE INT,
Address Varchar (255),
payment - Info Varchar (255)
foreign key (user - ID) REFERENCES
USER (USER - ID).
);
-- Admin Table
create Table admin (.
Admin - ID Int primary key.
user - ID INT,
Role Varchar (50).
foreign key (user - ID) References User (user - ID)
);

(1)

Perform SQL Relations Using DDL and DCL commands

DDL (Data Definition Language):

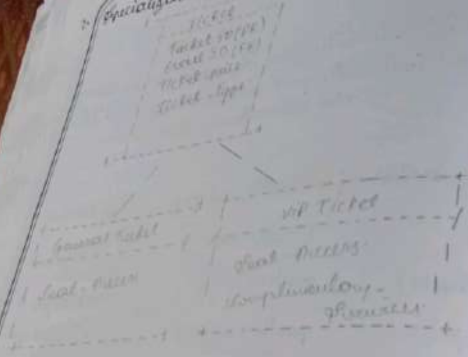
create user table (Generalization).

```
create table User (.
  User-ID Int primary key,
  first-Name Varchar (50),
  last-Name Varchar (50),
  Email Varchar (100),
  contact-Number Varchar (15),
  password Varchar (100),
  user-Type Varchar (10) - 'Admin' or 'customer'
);
```

- Specialized customer Table.

```
create table customer (.
  customer-ID Int primary key.
  User-ID INT.
  AGE INT,
  Address Varchar
  payment
```

Specialization Diagram: Ticket as superclass for general Ticket & VIP Ticket.



- Temple Table.

CREATE TABLE TEMPLE (.
Temple - ID Int primary key,
Name Varchar (100),
location Varchar (100),
Description Text.
);

Event Table.

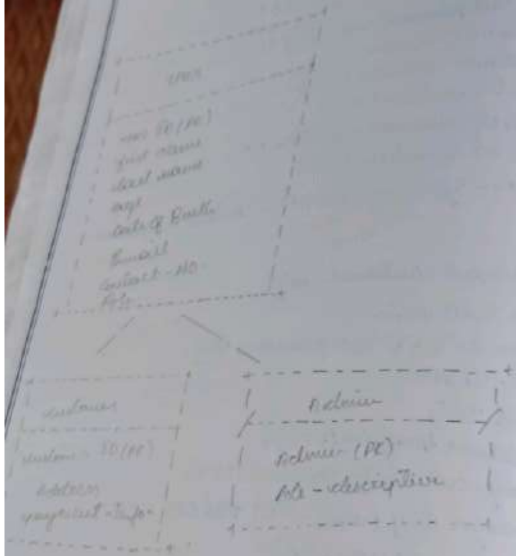
CREATE TABLE event (
Event - ID Int primary key.
Temple - ID Int.
Name Varchar (100),
Start - date DATE.
END - DATE . DATE.
foreign key (Temple - ID).
REFERENCES Temple (Temple - ID)
);

verified

- Ticket Table with Specialization handled by Ticket - Type attributes

Create Table Ticket (.
Ticket - ID Int primary KEY.
EVENT - ID INT,
TICKET - Type VARCHAR (20), - 'GEN
'VIP'
Ticket - Price Decimal (10,2)
Foreign Key (Event - ID) REFE
EVENT (Event - ID).
);

Generalization Diagram: User as Superclass for customer and admin.



- Booking Table (Surplus relation for Many-To-many)

```
CREATE TABLE Booking(
  Booking-ID INT primary KEY.
  customer-ID INT.
  Ticket-ID INT
  Booking-DATE DATE,
  Seat-Number-INT,
  foreign key (customer-ID)
  REFERENCES customer(customer-ID)
    FOREIGN KEY (TICKET-ID)
  REFERENCES Ticket (Ticket-ID)
  );
```

31/7/25

Result:

Thus, the hierarchical and network models have been successfully created for the Temple Tic[ket] online Booking management System, wi[th] appropriates Inheritance, relationship type[s], domain constraints, renamed relation[s] and SQL Implementations Using DDL a[nd] DCL commands.

VEL TECH - CSE	
EX NO	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	4.8
TOTAL (20)	17.8
[SIG]N WITH DATE	[signature]

- Booking Table (Supplies relation for Many-To-Many)

```
CREATE TABLE BOOKING(.
  Booking - ID INT Primary KEY.
  Customer - ID INT.
  Ticket - ID INT
  Booking - DATA DATE,
  Seat - Number - INT,
  foreign key (customer - ID)
  REFERENCES customer (customer - ID)
    FOREIGN KEY (TICKET - ID)
  REFERENCES Ticket (Ticket - ID)
  );
```

31/7/25

Result:

Thus, the hierarchical and network models have been successfully created for the Temple Ticket online Booking management System, with appropriates Inheritance, relationship type domain constraints, renamed relation and SQL Implementations Using DDL and DCL commands.

VEL TECH - CSE	
EX NO	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	7.5
TOTAL (20)	13.5
SIGN WITH DATE	

Date: 07/08/25

Task:3: Using clauses Operators and functions for Queries.

* Aim: To perform Query processing on a Temple Ticket online booking management system for different Retrieval result of Queries using DML, DRL Operations with aggregate functions, data functions, string functions set clauses, and operators.

* Temple:

Temple ID	NAME	LOCATION	CONTACT-NO
TID01	Tirumala Venkateshwara	Tirupati	902427695
TID02	Meenakshi Amman	Madurai	798922272
TID03	Kashi Viswanath	Varanshi	701398271
TID04	Jagannath Temple	puri	97012244
TID05	Golden Temple	Amritsar	99088164

O/P

Visitor:

Visitor ID	FNAME	LNAME	AGE	Email	Contact NO
V001	Anil	Kumar	30	anilkumar@gmail.com	9014276[illegible]
V002	Frank	Reddy	25	frank@gmail.com	970224[illegible]
V003	priya	Sharma	28	[illegible]@gmail.com	990829[illegible]
V004	Karan	patel	35	karan@gmail.com	7013982[illegible]
V005	Sneha	Rao	22	Sneha@gmail.com	7989222[illegible]

* Booking:

Booking ID	Temple I.D	Visitor I.D	Booking date	Ticket type	Am[illegible]
B001	T1D01	V001	2024-06-15	ViP	50[illegible]
B002	T1002	V002	2024-06-16	G.L	10[illegible]
B003	T1D01	V003	2024-06-18	G.L	10[illegible]
B004	T1D03	V004	2024-06-17	ViP	70[illegible]
B005	T1D01	V005	2024-06-19	G.L	2[illegible]

verified

Visitor:

Visitor ID	FNAME	LNAME	AGE	Email	Contact N
V001	Anil	Kumar	30	anilkumar@gmail.com	9010
V002	[illegible]	Reddy	25	[illegible]@gmail.com	9701
V003	priya	Sharma	28	[illegible]@gmail.com	990
V004	Aarav	patel	35	aarav@gmail.com	701
V005	Sneha	Rao	22	sneha@gmail.com.	798

Booking:

Booking ID	Temple I.D	Visitor I.D	Booking date	Ticket Type
B001	T1D01	V001	2024.06.15	V
B002	T1002	V002	2024-06-16	G
B003	T1D01	V003	2024.06.18	G
B004	T1D03	V004	2024.06.17	
B005	T1D01	V005	2024.06.19	

Priest:-

Priest	FNAME	LNAME	AGE	EMAIL	CONTACT
P001	RAMESH	Iyear	50	ramesh@gmail.com	9014276[illegible]
P002	Suresh	Sharma	45	Suresh@gmail.com	9701220[illegible]
P003	Manish	Roy	40	Manish@gmail.com	7989222[illegible]

2. Retrieve details of Visitors who first name starts with 'a'

Select *
from Visitor
where FNAME like 'A%';

Result;

Visitord ID	FNAME	LNAME	AGE	contact NO.
V001	Akash	Reddy	25	9701224401
V004	Aaran	patil	35	7018982716

Add a column for special-server in booking table.
Alter Table booking ADD Spective Beva Varchar(50);

Result:-
Tabel actered successfully.

Poriest :-

Poriest	FNAME	LNAME	AGE	EMAIL	CONTACT NO
P001	RAMESH	Iyear	50	ramesh @gmail .com	9014276925
P002	Suresh	Sharma	45	Suresh @gmail .com	9701224401
P003	Manish	roy	40	Manish@ gmail.com	7989222

Retrive details of Visitors who first name Starts with 'A'

Select *
from visitor
Where fNAME like 'A%';

Result;

VisitorID	FNAME	LNAME	AGE	contact NO.
V001	AKash	Reddy	25	9701224401
V004	Aarav	patel	35	7018982716

Add a column for Special-server Ine booking table.
Alter Table booking ADD Spective Beva Varchar(50)

Result :-
Tabel actered Successfully.

4. Count the no. of VIP Ticket booking.

Select count (*)
from Booking
where Ticket Type = 'VIP';

Result:
count (*)

5. Display Temple detail to Temple IDs. 'TID01' 'TID03' & 'TID04'

Select *
from Temple
Where TempleId IN ('TID01', 'TID03', 'TID04');

Result;

Temple ID	Name	location	contact. NO.
TID01	Trimula	Tirupati	9401224404
TID02	Kashi	Varanasi	7013982416
TID03	Golden Temple	Amritsar	7989222425

. Select Visitor ID & names at Visited who booked 'Special Ticket'

Select Visitor ID, FNAME, LNAME
from Visitor
Where Visitor ID IN(

4. Count the no of VIP Ticket booking.

select count (*)
from Booking
where Ticket Type = 'VIP';

Result:
count (*)

5. Display Temple detail to Temple IDS. 'TID01' 'TID03'
'TID04'

select *
from Temple.
where TempleId IN ('TID01', 'TID03', 'TID04');

Result;

Temple ID	Name	location	contact. NO
TID01	Trimula	Tirupati	940122444
TID02	Kashi	Varanasi	7013982
TID03	GoldenTemple	Amritsar	79892

Select Visitor ID & names at Visited who booked Ticket'

Select Visitor ID, FNAME, LNAME.
from Visitor
Where Visitor ID DNC.

select visitor ID from Booking where Ticket_Type = 'Special';
);

* Result;

Visitors	FNAME	LNAME
V005	Sneha	Rao

7. find the priest ID of priest who have not been assigned any Temple.

select priest ID
from priest
where Temple ID is Null;

Result;
priest ID
(No Result If all priests are assigned);

VEL TECH - CSE	
EX NO	8
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	2
RECORD (5)	[illegible]
TOTAL (20)	[illegible]
SIGN WITH DATE	[illegible]

[illegible] 7/18

Result: Thus, Query processing for Temple Ticket online booking management system using clauses, operators, and functions was successfully performed.

Select Visitor ID from Booking where Ticket, type = Speed pal;

);

Result;

Visitors	FNAME	LNAME
V005	Sneha	Rao

Find the priest ID of priest who have not been assigned any Temple.

Select priest ID.
from priest.
where Temple ID is Null;

Result;
priest ID.
(No Result If all priests are assigned).

VEL TECH - CSE	
EX NO	3
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	5
TOTAL (20)	18
SIGN WITH DATE	

18

Result: Thus, Query processing for Temple Ticket online booking management system using clauses, operators, and functions was successfully performed.

Date: 14/08/25 Task: 4)

Using function for Queries & Writing Sub Queries.

* Aim: To perform advanced Query processing and Test its heuristics by designing optional corrected and nested subqueries such as finding summary statistic for the Temple Ticket online

4.1: To retrieve all Temple details, Including the count of booking for Each Temple.

Select t. Temple ID,
t. Name As Temple Name.
t. location.
t. contact-NO.
Count (b. booking ID) As Total Booking
from Temple t.
Left Join Booking b.
On. t. Temple ID = b. Temple I.D.
Group By T. Temple ID, T Name, t. location
constant - NO;

Date: 11/08/25 Task: 4)

Using function for Queries & Writing sub Queries.

Aim: To perform advanced Query processing and test its heuristics by designing optional corrected and nested subqueries such as finding summary statistic for the Temple Ticket online

4.1: To retrieve all Temple details, Including the count of booking for Each Temple.

Select t. Temple ID,
t. Name As Temple Name.
t. location.
t. contact-NO.
Count (b. booking ID) As Total Booking
from Temple t.
Left Join Booking b.
on. t. Temple ID = b. Temple I.D.
Group By T. Temple ID, T Name, t. location
contact - NO;

Output:

Temple ID	Temple NAME	Location	Total Booking
TI001	Tirumala	Tirupathi	3
TI002	Meenakshi Amman	Madurai	1
TI003	Kashi	Varanasi	1
TI004	Jagannath Temple	puri	0
TI005	Golden Temple	Amritsar	0

TP

4.2: To retrieve the total no of special seva bookings in a Temple-wise manner.

Select t.NAME As Temple Name.
count (*) As Total Special Booking.
from Temple t.
Join Booking b.
ON t.Temple ID = b.Temple ID.
where b.Ticket-Type = 'Special'
Group By t.Name;

* Output :

Temple Name	Total Special Booking
Tirumala Venkateshwara	1

u: 3: To Retrieve the details of Temples where booking. Include 'VIP' Ticket

```
Select *
from Temple.
Where Temple ID IN (
  Select Temple ID.
  from Booking.
where Ticket-Type = 'VIP'
);
```

Output :

Temple ID	Name	location	contact-NO
T1D01	Tirumala	Tirupati	9701224404
T1D03	Kashi	Varanasi	7013982716

Output :

Temple Name	Total Special Bo
Tirumala Venkateshwara	1.

1: 3: To Retrives the details of Temples where booki
Include 'VIP' Ticket

```
Select *
from Temple.
Where Temple ID IN.
  Select Temple ID.
  from Booking.
Where Ticket_Type = ('VIP'
;
```

Output :

Temple ID	Name	location	contac
1D01	Tirumala	Tirupati	97012
T1D03	kashi	Varansi	7013

4.4: To Retrive Visitors and booking details wit visitors who are above 25 years old.

```
Select v. Visitor ID.
       v.F Name As Visitor Name,
       V. age,
       v. Booking ID.
       b. Booking - Date,
       b. Ticket - Type.
       b. amount.

from Visitor
Join Booking b.
on Visitor ID = b.Visitor ID.
where V.Age = 25;
```

Output;

Visitor ID	Visitor Name	Age.	Booking	Amount.
V001	Anil	30	B001	500
V003	priya	28	B003	100
V004	Aarav	35	B001	700.

.5: To retrive the details of temples with no bookin

```
Select *
from Temple.
  Where. TempleId NOT. IN (.
```

4.4: To Retrieve Visitors and booking details wit.
Visitors who are above 25 years old.

Select v. Visitors ID,
v.F Name As Visitors Name,
V. age,
v. Booking ID.
b. Booking. Date,
b. Ticket - Type.
b. amount.

From Visitors
Join Booking b.
on Visitor I.D= b. Visitors I.D.
where V.Age = 25;

Output;

Visitor ID	Visitors Name	Age.	Booking	Amount
V001	Anil	30	B001	500
V003	priya	28	B003	100
V004	Aaraa	35	B001	700.

5: Two retrieve the details of temples with no Booki

Select *
From Temple.
Where. TempleId NOT. INC.

Select Temple ID.
from Booking
);

Output ;

Temple ID	NAME	Location	Contact-NO.
TI004	Jagannath Temple	puri	9701224404.
TI005	Golden Temple	Amritsar	9014276954.

4.6 : To retrieve the templeid, Temple name, and visitor name for a particular Visitor Given.

```
Select t. Temp ID,
    t. Name As Temple Name.
    V. FName As Visitor Name.
 from Temple . T.
 Join Booking b.
on t. Temple ID = b. Temple ID.
 Join Visitor. V.
ON b. Visitor ID = V. Visitor ID.
 Where V. Visitor ID = 'V005';
```

Select Temple ID.
from Booking.
);

Output;

Temple ID	NAME	Location	Contact-NO.
TID04	Jagannath Temple	puri	9701224404.
TID05	Golden Temple	Amritsar	9014276954

6:- To retrieve the templeid, Temple name, and Visitor name for a particular Visitor given.

elect t. Temp ID,
t. Name As Temple Name.
v. FName As Visitor name.

from Temple. T.
Join Booking b.
n t. Temple ID = b. Temple ID.
Join Visitor. V.
N b. Visitor ID = V. Visitor ID.
Where V. Visitor ID = 'V005';

Output :

Temple ID	Temple NAME	Visitors NAME
T3001	Tirumala	Sasha

VEL TECH - CSE	
EX NO	4
PERFORMANCE (5)	[illegible]
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	5
TOTAL (20)	14+5=19
SIGN WITH DATE	

4/8

Result : Thus the Queries Using Function Joins and nested Subqueries were Successfully Executed for the Temple online Booking management System.

Output :

Temple ID	Temple NAME	Visitors NAME
T I D 01	Tirumala	Sushin

VEL TECH - CSE	
EX NO	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	14+5=19
TOTAL (20)	19
..N WITH DATE	

4/8

... The Queries Using Function Twice and ... successfully Executed for ...

Task 5 date: [illegible]

Writing Join Queries, Equivalent and Recursive Queries

(Tool: SQL | Oracle, [illegible])

* Aim: To perform advanced Query Processing and test its heuristics using Join Queries, Equivalent Queries, and Recursive Queries for the Temple Ticket Online Booking management System which manages devotee details, poojas, Tickets and booking details.

Queries & Outputs

5.1. To Retrieve all Temples and their poojas

Sql:

```
Select T.Name As Temple, P.pooja Name, P.price.
from Temple T.
Join pooja P on T.Temple ID.
   = P.Temple ID.
```

5.2. To list all booking along with the devotee and poojas details.

Sql:

```
Select b.Booking ID, d.F Name As Devotee, p.pooja Name.
 b.Booking date b.Slottime, b.Status.
from booking b.
   Join Devotee d on b.Devotee.ID.
      = d.devotee ID.
```

Task 13 date 11/08/25

Writing Join Queries, Equivalent and for recursive queries

(Tool: SQL / Oracle, MLM / Flapper).

Aim: To perform advanced Query Processing and test its heuristics using Join Queries, Equivalent Queries, and Recursive Queries for the Temple Ticket Online Booking management system which manages devotees, temples, poojas, tickets and booking details.

Queries & Outputs

5.1. To Retrieves all Temples and their poojas

Sql:

```
Select T.Name As Temple, P.pooja Name, P.price.
from Temple T.
Join pooja P on T.Temple I.D.
    = P.Temple ID.
```

5.2. To list all booking along with the devotes and pooja details.

Sql:

```
Select b.Booking ID, d.F Name As. Devotee; p.pooja Name
    b.Booking date b.Slottime, b.Status.
from booking b.
    Join Devotes d on b.Devotes.I.D.
        = d.devotes ID.
```

output 01

Temple	pooja Name	price
Tirupati temple	Suprabhatam Seva	200.
Tirupathi temple	Archana	150.
Kanchi Temple	Abhishekam	500
Madurai	Special Darshan	250.

output 02

Booking ID	Devote	pooja Name	Booking
B101	Rajesh	Suprabhatam Seva	22-6-23
B102	Meena	Archana	22-6-23
B103	Arjun	Abhishekam	23-6-23
B104	Kavitha	Special darshan	23-6-23
B105	Suresh	Abhishekam	24-6-23

Output :-03

Devotee	Total booking
Rajesh	1.
Meena	1
Arjun	1
Kavitha	1
Suresh	1

output 07

Temple	pooja Name	price.
Tirupati temple	Suprabhatam Seva	200.
Tirupathi temple	Archana	100.
Kanchi Temple	Abhishekam	500
Madurai	Special Darshan	250.

output 02

Booking ID	Devote	pooja Name	P[illegible]
B101	Raju[illegible]		

Output : 04.

Devotee	Fname	Mobile No.
D102	Arjun	9876543210
D105	Suresh	8765432190

Output : 05.

Pooja Name	Total booking.
Suprabhata Seva	1.
Archana	1.
Abhishekam	2.
Special darshanam	1.

Output : 04·

Devotee	Fname	Mobile NO.
D102	Arjun	9876543210
D105	Suresh	8765432190

Output : 05·

Pooja Name	Total booking.
uprabhata Seva	1.
chana	1.
shekam	2.
darshanam	1.

O/P

Join pooja p ON b.pooja ID
= p.pooja ID;

5.3. Count the no of booking made by each devotee.

SQL:
select d.f name as devotee, count (b.booking ID) as total
as total booking from devotee D.
LEFT Join Booking b on d.devotee ID = b. Devote ID.
Group By d.f Name;

5.4 To find all devotee who booked 'abhishekam'

SQL:
select d. devotee ID, f Name, d. Mobile NO.
from devotee d.
Join booking b on N. Devote.
ID = b. devotee ID.
Join pooja ID on b. pooja ID = p. pooja ID.
where p. pooja name = 'Abhishekam'.

5.5. To retrieve all poojas and the no. of Times they were
booked.

SQL:
SELECT p. Pooja Name, count (b. booking ID) as Total
booking
from pooja p.

Join pooja p ON b.pooja ID.
= p.pooja ID;

5.3. count the no. of booking made by each devotee.

sql:
select d.f name as devotee, count (b.booking ID) as total.
as total booking from devotee D.
LEFT. Join Booking b on d.devotee ID = b. Devote ID.
Group by d.f Name;

5.4 To find all devotee who booked 'Abishekam'.

sql!
select d. devotee ID, f Name, d. Mobile No.
from devotee d.
Join Booking b on N. Devote.
ID= d.devotee ID.
Join pooja ID on b.pooja ID = p.pooja ID.
where p.pooja name = 'Abishekam';

5.5. To retrieve all poojas and the no. of Times they are
booked.

sql:
SELECT P. Pooja Name, count (b.booking ID) as Total
booking
from pooja p.

LEFT Join booking b on p. poojas b. pooja ID.
Group By p. pooja Name;

5.6 To Retrives the total number of cancelled booking Temple wise

Sql:
Select T. Name as Temple, count (b. Booking ID) as cancelled booking from Temple T.
Join booking b on T. Temple ID = b. Temple ID
where b. Status = 'cancelled'
Group By T. Name.

5.7: To retrive Temple details where booking were successfully

Sql:
Select Distance T. Temple ID, T. Name, T. location
T. lead priest.
from Templet.
Join Booking b on Temple ID =
b. Temple ID
where b. Status = 'Confirmed';

5.8. To retrives devotes and booking details for devotes above 50 years old.

Sql:
Select d. FName, d Age, b. booking ID, b. booking date,
b. slottime b. Status.

LEFT Join booking b on p.poojaID b. Pooja ID.
Group By p.pooja Name;

5.6. To Retrieves the total number of cancelled booking Temple wise

Sql:
Select T.Name as Temple, count (b. Booking ID) as.
cancelled booking from Temple T.
Join booking b on . T. Temple ID = b. Temple ID
where b. Status = 'cancelled'
Group By T. Name.

5.7: To retrieve Temple details where booking were successfully

Sql :
Select Distance T. Temple . ID. T.Name , T. location
T. lead priest .
from Templet .
Join Booking b on . Temple ID =
b. Temple ID
where b. Status = 'Confirmed';

5.8 . To retrives devotes and booking details for devotes a
50 years old.

Sql:
Select d. FName, d Age, b. booking ID, b. booking date
b. slottime b. Status.

Temple.	Cancel Booking.
Kanchi Temple	1.

Temple ID	Name	location	head priest
T001	Tirupathi temple	Tirupati	Ramanujam
T002	Kanchitemple	Kanchipuram	Narayana Das.
T003	Madurai Temple	Madurai	Subramaniam

Temple	Cancel Booking
Kanchi Temple	1

Temple ID	Name	Location	Head priest
T001	Tirupathi temple	Tirupati	Ramanujan
T002	Kanchitemple	Kanchipuram	Narayan Das.
T003	Madurai Temple	Madurai	Subramaniam.

[illegible]

[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

[illegible]

[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]

Temple	pooja Name	Devotee	Bookingdate	Slot Time
Tirupathi Temple	Archana	meena	22-July-2023	9:00am

Temple	pooja name	Devotee	Booking date	Slot Time
Tirupathi Temple	Archana	meena	22-July-2023	9:00am

E.NO	[illegible]
PERFORMANCE (3)	[illegible]
RESULT AND ANALYSIS (3)	[illegible]
VIVA VOCE (2)	[illegible]
RECORD (2)	[illegible]
TOTAL (10)	[illegible]
SIGN WITH DATE	[illegible]

:

Thus the Temple ticket online Booking management Simple Querying using Join Queries Equivalent Queries and recursive Queries were executed successfully to manage temple Donors, priests, pooja and booking.

Task: 6

TASK-6

Date: 22-8-25

procedures, functions and loops using PL/SQL on Number theory and business scenarios.

* Aim: To write and Execute PL/SQL procedures, functions and loops on number theory and business scenarios.

* Algorithm:

A. Number Theory Example: prime number check using procedure

1. Start.
2. Read a Number n.
3. Initialize counter.
4. use loop for check distribution
5. If divisible → not prime, Else.
6. display result.

B. Business Scenario Examples Salary bonus calculation using function

Start.

Create a function that accepts Emp. Salary.

If Salary < 2000 → 20% bonus If salary b/w 2000 - 50000 → 10% bonus.

Else → 5% bonus.

Return final salary with bonus.

End.

Program

```
Number Theory - prime Number check Set
create or Replace procedure
    check-prime (N/N Number IN) is.
flag Boolean := True;
Begin
If n<=1 Then
DBMS-Output.put-line(N||'Is not prime');

Else
for i in 2..n/2 loop
    If MOD(n,i) = 0 Then
      flag := false;
       Exit;
    End loop;
If flag then
DBMS-Output.put-line
                    (N||'Is prime');

Else
  DBMS-Output-put-line (n||'is Not prime');

    END If;
    END If;
    END;
-- Execution
Begin
     check-prime (29);
     check-prime (20);
       END;
       /
```

Program.

1) Number theory - prime Number check Set

create or Replace Procedure.

check - prime (N/N Number IN) is.

flag Boolean : = True ;

Begin.

If n<=1 Then.

DBMS - Output . put . line (N || I Not prime :) ;

Else.

for i In 2 . n/2 loop.

If MOD (n,i) = 0 Then .

flag : = false ;

Exit ; .

End loop :

If ~~fo~~ flag Then.

DBMS - Output . put - line .

Else. (N || Is prime :) ;

DBMS - Output - put - line (n || . Is Not prime) .

END If :

END If .

Output

for prime Number check:

29 is prime:

20 is not prime

for Bonus Calculation:

final salary with Bonus: 13400

final salary with Bonus: 33000

* Output

For prime Number check:

29 is prime.

20 is not prime.

For Bonus Calculation:

Final Salary with Bonus: 13400

Final Salary with Bonus: 33000

* Business Scenario: Employee Bonus calculation. Set serveroutput on;

```
create or replace function calc_bonus (salary Number)
Return Number is
bonus Number;
Begin
  If salary < 2000 Then
    bonus := salary * 0.20;
  Else if salary between 2000 and 5000 Then
    bonus := salary * 0.10;
  End If;
  Return (salary + bonus);
End;
/
-- Execution
declare
  final_sal := calc_bonus(3000);
  DBMS_Output.put_line ('final salary with
End;
/
```

Result: Thus PL/SQL procedures, functions and loops were successfully implemented for various library and business operations.

b. Business Scenario - Employee Bonus calculation. Set serveroutput on;

```
create or Replace function calc_bonus (salary Number)
Return Number is
bonus Number.
Begin
  If salary < 2000 Then.
    bonus = salary 0.20;
  Else if salary between 20000 and 50000 Then.
    bonus : = salary * 0.10;
  End If;
  Return (salary + bonus);
End;
-- Execution
declare
  final_sal := calc_bonus (3000);
  DBMS_Output.put_line (final salary
End;
/
```

Result: Thus PL/SQL procedure function was successfully implemented for number, business scenarios.

TASK: 7 Date: 11/03/25

Triggers using Event Exceptions

- Aim: To conduct Event views and Exceptions handling SQL operations for the Temple Ticket online booking management

- Trigger:

Requirement: When a new booking is inserted in to the booking Tables, automatically Issue an Ticket.

Sql:

```
create or replace Trigger
Trg-Insert-Ticket
After Insert on booking
    for Each Row.
  Begin
      Insert Into Ticket
  (Ticket ID, Booking ID, QR code, Status)
      Values (
  Seq Ticket Nextval
   NHN Booking ID;
   ION Know;
       Null;
       Null;
     Sys-guid ( ).
         'Active'
       );
     End;
     )
```

TASK 7 date 11/09/25

Triggers, using [illegible] Exceptions

Aim: To conduct Event Views and Exceptions handling [illegible] operations for the Temple Ticket [illegible] Booking [illegible]

Trigger:

Requirement: When a new booking is inserted in to the booking Tables, automatically [illegible]

SQL:

```
create or replace Trigger
Trg-Insert-Ticket
After Insert on booking.
    for Each Row.
  Begin
      Insert Into Ticket.
  (Ticket ID, Booking ID. QR code', Status)
        Values (.
  Seq Ticket Inbert.
   NHN Booking ID;
    ION know;.
        Null;
         Null;
       Sys-guid ( ).
            'Active'.
           ) ;
         End;
        ).
```

(27)

Out - Given - booking - Ids :
Type - be I - Number list ( );
for active
Select - Booking ID
from Booking
where Temple ID = In - Temple - it
And count (Booking ID) : 0
End loop;
End;
)

* Execution Block :

DECLARE
Temple - Id Number : - 101 ;
Given - booking - id type - expression - BER list ;

Begin
Get Given booking ID's for Temple :
Temple - Id Given - booking - id);
for (IN. - - - Given - ids , count loop DBMS - Output ('Given booking ID:) / Given - booking)

Begin
Get Given booking ID's for Temple:
Given - booking - Id);
11 Given - booking - id's (i))
End loop);
END;
)

EX NO	[illegible]
PERFORMANCE (5)	[illegible]
RESULT AND ANALYSIS (5)	[illegible]
VIVA VOCE (5)	[illegible]
RECORD (5)	[illegible]
TOTAL (20)	18
SIGN WITH DATE	[illegible]

24/9/24

Result : Thus, the figures Views and Exceptions for the Temple Tic[illegible] online booking Management System were successfully Implemented and Verified.

Out - Even - booking - Ids: = -
Sys - be I - Number - list ( );
for in live
Select Booking ID
from Booking
where Temple ID = In - Temple - it
And cursed (Booking ID): 0
End loop;
End;
)

Execution Block:
DECLARE
Temple - Id Number: - 101;
Even - booking - id sys. operium - BR list;
Begin
get Even booking IDs for Temple (
Temple - Id Even - booking - id);
for (IN, - - - Even - ids, count loop DBMS - Out
booking ID:) / Even - booking)
Begin
yet Even booking ID's for Temple
Even - booking - Id);
11 Even - booking - id's (i))
End loop);
END;
)

27/9/25

Result: Thus, the figures Views and Exception
online booking Management Implem
Implemented and Verified.

2. Output:

{"acknowledged": true, "insertedId":

objectId("651c1f726bebfe9993adf901")}

3. Output:

{"acknowledged": true,

"insertedIds": [

objectId("651c1f726bebfe9993adf902"),

objectId("651c1f726bfe993adf903"),

objectId("651c1f726bebfe9993adf904")

]

}

4. Output

{"_id": objectid("651c1f726bcbfe9a93adf901"), "Temple Name": "Sri Venkateswara [illegible]

"Booking ID": "B001", "Temple Name" ... "devotee Name": "Ramesh", [illegible]

"location": "Tirupati", "Booking date": "2025-09-10"}

9876543210, "Booking date"

"..." 9a93adf 902")

{"_id": object("651c1f726bebfe7a93adf902"), "Temple Name": "Meenakshi Temple"

"Booking ID": "B002", "Temple Name": "Meenakshi", "devotee Name": "Suresh", [illegible]

"location": "Madurai", "Booking date": "2025-09-12"}

9876501234, "Booking date": "2025-09-12"}

Output:

{"_id": object id("651c1f72bebfe7993adf903"),

"Booking ID": "B003",

"Temple Name": "Kapaleeshwarar Temple",

"location": "Chennai",

"Devotee Name": "priya",

Task: 8: Crud Operation Environment Detail

Aim: To perform crud using mongo db on online booking management system, database and performing crud operation Inserting, Querying, finding, updating

Steps:

Step: 1: Install mongo DB using the follow
https://www.mongodb.com/try/downloa

Step: 2: Install mongosh using the below
https://www.mongodb.com/docs/mongo
and - Install - mongosh

Step: 3: To add mongoDB shell binary's
Environment Variable.

3.1: open the control panel.
3.2: Go to system and security → syste
3.3: click advanced system setting
3.4: click Environment Variable.
3.5: In system Variable, select path
3.6: add the file filepath to your mon
3.7: click ok to confirm

Step: 4: Open a command prompt a
confirm Installation

Step: 5: Open mongo shell 4.0 form c
files / mongo DB / Server / bin / m

Step: 6: Type the crud (create, Read
for mongosh.

...ies

...ucl.

...ft.c

...fu

Crud Operations:

Create collection:

db.createcollection("Temple Booking").

{"OK":1}.

Insert One Record.

db.TempleBooking.InsertOne(.

{BookingID:"B001", Templename:"Sri Venkat

Swami Temple", location:"Tirupati", devote

Name:"Ramesh", phone:9876543210, booking

date:"2025-09-10"}

}

Insert Many Records.

db.TempleBooking InsertMany([.

{BookingID:"B002", TempleName:".Meenakshi.

Amman Temple", location:"Madurai",.

8.3:- Query All Temple:

{

"Id": object Id(651d1c9e66bfe7993adf4010

"TempleID":"T1T01"

"Name": Meenakshi Temple.

"location": Meenakshi Temple.

"Contact": 9876T03Q10.

"Timing": 6:00AM TO 9:00.PM".

```
{
  "_id": objectId("65 1d1cc36ebb fe 7773 adf 011"),
  "Temple id" : "T1001"
  "Name" : "Ramanathaswamy Temple"
  "location": Rameshwaram
  "Timing": "5:00AM to 8:00PM"
}
  "id": object ID("65 1dcc36ebb fe7779 adf 012");
  "Temple ID": "T1003";
  "Name": "Sri Kashi Vishwa Temple"
  "location": "Tamil nadu";
  "contact": 9687594321
  "Timing" : 7:00 Am to 8:00PM.
```

* 8.4: Query Ticket Booking by user name:

```
db.Tickets.find({user name: "Tulasi"}).
```

* 8.5: update Temple contact:

```
db.Temple.update one(
  {Temple ID: "T1001"}
  {$set: {contact: 9982321879}}
);
```

* 8.6: Delete a ticket booking:

```
db.Ticket.delete one({
  Ticket ID: "Tic 1009"}).
);
```

{
"_id": objectId("65 1d1cc 36c6b fe 9993 adf 0a"),
"Temple id" : "T1002"
"Name" : "Ramanathaswamy Temple"
"location" : Rameshwaram
"Timing" : "5:00Am to 8:00Pm"
{
"id" : object ID("651dcc36c6b fe4993adf0a"
"Temple ID" : "TID03";
"Name" : "Shri kashi vishwanath Temple"
"location" : "Tamil nadu";
"contact" : 9687594321
"Timing" : 7:00 Am to 8:00Pm.

8.4: Query Ticket Booking by user name.
db.Tickets.find({user name : "Tulsi"

8.5: update Temple contact:
db.Temple.update one(
{Temple ID: "TID01"}
{$set:{contact : 9982321879}}
);

8.6: Delete ~~a~~ Ticket booking:
db.Ticket.delete one({
Ticket ID: "TIC TD09"});
);

Aim: To perform CRUD operations like creating, inserting, querying, updating and deleting on Neo4j graph for an online temple ticket booking management system.

Step 2: Set up Neo4j [illegible]:

Open:
[illegible] → [illegible] with [illegible]
Copy the [illegible] password [illegible] and login.
The [illegible]
[illegible] Queries in the Neo4j browser

Create nodes with properties:
Each node represents an entity like [illegible], Temple, Booking
Create [illegible] node:
create (d:[illegible] {[illegible] ID: '0001', na

Create a temple node:
create (t:Temple {Temple ID: 'T001', Name

Create Booking node:
create booking node {Booking ID: '0001', [illegible]},

Create Relationship
Devotee makes booking:
Match (d: Devotee {[illegible] ID: '0001'}), (
create (d)-[:MADE]->(b)
Return d, b.

Booking linked to Single

Match (b: Booking {Booking ID: '[illegible]'}), (c

[illegible] (b) - [illegible] - (c)

Return b, c

Display all [illegible]:

Match (n) Return (n).

Retrieve particular Booking details:

Match (b: Booking {Booking ID: 'B001'}) Re

Update particular [illegible] details:

Match (d: [illegible] {[illegible] ID: 'D001'})

[illegible] = '9123456780'; d.exp = 31.

Return d.

Delete Particular Booking.

Match (b: Booking {Booking ID:

('B001'}) delete b.

5/1/25

Result: By following the above steps was [illegible]
Inserted, Queries updated and deleted
Booking Management.

VEL TECH
EX NO
PERFORMANCE
RESULT AND AN
VIVA VOCE
RECORD
TOTAL

[illegible]

[illegible]

[illegible] Tax 2015

[illegible] expenditure [illegible]

[illegible] ID → [illegible] (Transaction expenditure)

[illegible] ID → [illegible] (Transaction expenditure)

[illegible] ID → [illegible] (Transaction expenditure)

[illegible] Payment ID – payment method – payment status

Found it (closure of candidate key):

Booking ID* {all attributes} → confirms Booking ID is candidate key.
User ID, Temple ID, Priest ID, Ticket ID, payment ID also act as foreign key with their own dependencies.

* Step (c): minimal cover / canonical cover.
we reduce FDs to minimal form:
User ID → UName, UEmail, UContact
Temple ID → TName, TLocation
Priest ID → Priest Name, Priest contact
Ticket ID: Ticketing Type, price
Payment ID → payment Mode, payment status, Transaction date
Booking ID → User ID, Temple ID, Priest ID, Ticket ID, Booking date, darshandate, Timeslot, payment ID.

* Step (d): Normalize to.
condition for 2NF:
* Must already be in 1NF.
- No partial dependency (no prime attributes depending only on part of a composite key).
Booking ID is the primary key (not composite)
So no partial dependency exists
The schema is in 2NF.

Step (e): Normalize to 3NF
condition for 3NF
Must already be in 2NF
No transitive dependencies (non-prime attributes dependency on non-prime)
Booking ID → User ID → UName (Transitive dependency)
Booking ID → Temple ID → TName, TLocation (Transitive dependency)
Booking ID → Priest ID → Priest Name (Transitive dependency)
Booking ID → Payment ID - payment method - payment - status

[illegible]

VEL TECH	
EX NO	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	

Result:

[illegible]

[illegible]

[illegible]

1. [illegible]
2. [illegible]
3. [illegible]

[illegible]

1. [illegible]
2. [illegible]
3. [illegible]
   - [illegible]
   - [illegible]
   - [illegible]
   - [illegible]
   - [illegible]
4. [illegible]
   - [illegible]
   - [illegible]
5. [illegible]
6. [illegible]
   - [illegible]
   - [illegible]
   - [illegible]

   [illegible]
   - [illegible]
   - [illegible]

[illegible]

[illegible]

[illegible]

```
CREATE TABLE coupons (
id SERIAL PRIMARY KEY,
code VARCHAR(50) UNIQUE,
type VARCHAR(10),          -- 'percentage' or 'fixed'
value NUMERIC,             -- percent or fixed amount
max_discount NUMERIC NULL,
min_order NUMERIC DEFAULT 0,
expiry DATE,
active BOOLEAN DEFAULT TRUE,
usage_limit INT DEFAULT 0,   -- 0 = unlimited
per_user_limit INT DEFAULT 1

);

CREATE TABLE coupon_usage (
id SERIAL PRIMARY KEY,
coupon_id INT REFERENCES coupons(id),
user_id INT REFERENCES users(id),
used_count INT DEFAULT 0

);

CREATE TABLE payments (
id SERIAL PRIMARY KEY,
user_id INT REFERENCES users(id),
coupon_id INT NULL REFERENCES coupons(id),
amount NUMERIC,
discount NUMERIC,
final_amount NUMERIC,
status VARCHAR(20), -- 'success'|'failed'
method VARCHAR(50),
created_at TIMESTAMP DEFAULT now()
```

1.);
Find coupon:

SELECT * FROM cou, ns WHERE code = 'WELCOME10';

2. Check total usage:

SELECT COALESCE(SUM(used_count),0) AS total_used
FROM coupon_usage WHERE coupon_id = 123;

3. Get user's usage:

SELECT used_count FROM coupon_usage WHERE coupon_id = 123 AND user_id = 45;

4. Record successful payment:

INSERT INTO payments (user_id, coupon_id, amount, discount, final_amount, status, method)
VALUES (45, 123, 1000, 100, 900, 'success', 'card');

5. Increment user coupon usage (insert or update):
6. INSERT INTO coupon_usage (coupon_id, user_id, used_count)
7. VALUES (123,45,1)
8. ON CONFLICT (coupon_id, user_id) DO UPDATE
9. SET used_count = coupon_usage.used_count + 1;

10. **EXAMPLE: Concrete Coupon + Walkthrough**

11. Coupon row:

d	code	type	value	max_discount	min_order	expiry	usage_limit	per_user_limit
	WELCOME10	percentage	10.00	150.00	500.00	2025-12-31	1000	1

12. User places order:

- r_id = 45
- cart_amount = 1200.00
- coupon_code = WELCOME10

dation:

cart >= min_order (1200 >= 500) → OK

percentage 10% → raw discount = 120.0 → below max_discount(150) → discount = 120.00

final_amount before tax = 1200 - 120 = 1080.00

Assume tax 18% on final_amount (optional):

- tax = 1080 * 0.18 = 194.40
- payable = 1080 + 194.40 = 1274.40

Payment simulation success.

SAMPLE PROGRAM OUTPUT(Console-style)

Applying coupon: WELCOME10

User: 45 Cart: 1200.00

Coupon valid — percentage 10% (max 150.00)

Discount applied: 120.00

Amount after discount: 1080.00

Tax (18%): 194.40

Final payable amount: 1274.40

Payment status: SUCCESS

Payment ID: PAY_20251017_001

Coupon usage updated for user 45 (used_count = 1)

user_id=45

- coupon_code = WELCOME10

Validation:

- cart >= min_order (1200 >= 500) → OK
- percentage 10% → raw discount = 120.0 → below max_discount(150) → discount = 120.00
- final_amount before tax = 1200 - 120 = 1080.00

Assume tax 18% on final_amount (optional):

- tax = 1080 * 0.18 = 194.40
- payable = 1080 + 194.40 = 1274.40

Payment simulation success.

SAMPLE PROGRAM OUTPUT(Console-style)

Applying coupon: WELCOME10

User: 45 Cart: 1200.00

[illegible]

• Result: [illegible]

[illegible] successfully.